



Agentic payments on MOI

An agent that pays another agent.

Your agent finds a seller it has never met, checks who it is on chain, and pays it mid-request. No signup. No API key. No human.

Adithya · Ecosystem, Sarva Labs

Agents can find each other. They still can't pay each other.

Session 3 gave an agent an on-chain identity. But an identity that can't transact is a business card. The moment one agent wants something another agent has, three things break.

01

No way to charge

Every paid API assumes a human: a signup, a card on file, an invoice. None of that survives contact with an agent acting alone.

02

No way to know who

A payment address is 32 bytes. It doesn't tell you whose it is, or whether the listing was swapped a minute ago.

03

No sane unit

You cannot put a human in the loop on a tenth of a cent. The approval costs more than the thing being bought.

Today: two agents transact end to end — and the chain answers who is being paid.

The session, end to end

01 What x402 is

HTTP 402, reserved in 1997 and finally used. Four messages.

02 Where it stops

It moves value. It says nothing about who you're paying.

03 Discovery by skill

Find a seller in the registry. No URL handed to you.

04 The 402 response

Price, asset, and where to pay — machine readable.

05 The identity check

Is that address really the seller's? The one question x402 can't ask.

06 Pay and verify

A native MAS0 transfer, then a facilitator that reads the chain.

07 Watch it refuse

Swap the seller's address. The agent walks away.

08 What comes next

Authority in session 8. Conditional settlement in session 9.

x402 gives you an address. MOI tells you whose it is.

x402 is a payment protocol, and a good one. It deliberately says nothing about identity. That gap is exactly the shape of what MOI already provides.

X402 ANSWERS

- how much — 1 USDM
- which asset — a MAS0 asset id
- where to — 0x0000...97c4
- by when — a timeout

MOI ANSWERS

- + who owns that address
- + are they still active
- + what do they claim to sell
- + did the payment really land

Without the second column, you are wiring money to a number.

Four messages. That's the whole thing.

HTTP 402 Payment Required was reserved in 1997 and left unimplemented for 28 years. x402 finally uses it — and the server never touches a chain.

```
GET /book/the-prince           → ask
402 Payment Required { accepts: [ ... ] } → pay me this, here
GET /book/the-prince + X-Payment → retry, with proof
200 OK + summary + X-Payment-Response → goods + receipt
```

The server stays dumb

It declares a price and points at a facilitator. It never signs, never holds keys, never learns what a chain is.

The facilitator is the seam

The only chain-specific piece in x402. Adding MOI meant writing one service — not forking a protocol.

One purchase, thirteen steps, one second

- 1 Ask the registry — who sells books?
- 2 Read the catalog. Free — discovery shouldn't cost money.
- 3 The agent picks the book that answers its question.
- 4 Ask for it → 402 Payment Required

ON CHAIN

6 Is that address really the seller's registered wallet?

THE BEAT

- 7 Pay — the buyer moves its own funds.
- 8 Retry with the signed payment attached.
- 10 Facilitator runs nine checks. All read-only.
- 13 Book delivered. Receipt carries a real interaction hash.

ONLY WRITE

Exactly one on-chain write per purchase. Everything else is HTTP, or a read.

Nobody hands you a URL

The buyer searches the registry by skill. It learns the seller's agent id, its operating wallet, and where it runs — all from the chain.

```
discover.ts

const found = await discoverBySkill(registry, 'sells-books')

// straight off the chain
found[0].agent_id      // "agent_12"
found[0].agent_wallet  // 0x0000...97c4
found[0].url           // where it runs
found[0].status        // ACTIVE
```

BE PRECISE

It is not a search engine

An $O(n)$ client-side scan — every agent id, then every profile, then filter on the card's skill tags.

The registry has no index. Fine at demo scale. Don't call it semantic search.

A machine-readable bill

The seller answers 402 with everything an agent needs to pay, and nothing it doesn't. Field names are the x402 spec's, verbatim.

402 response body

```
{ "x402Version": 1, "accepts": [{  
  "scheme": "moi-transfer",  
  "network": "moi-voyage-devnet",  
  "maxAmountRequired": "1",  
  "asset": "0x...",           // a MAS0 asset  
  "payTo": "0x0000...97c4",   // ← 32 anonymous bytes  
  "extra": { "payToAgentId": "agent_12" } // ← our addition  
}] }
```

payToAgentId is the hook. It's what lets the buyer go and check.

The question x402 cannot ask

Before a single unit moves, the buyer goes back to the chain and asks whether that payment address really belongs to the agent it thinks it's buying from.

```
identity-check.ts

const onChain = await readAgentWallet(
  registry, requirements.extra.payToAgentId
)

if (onChain !== requirements.payTo) {
  return 'refuse — payTo is not their wallet'
}
```

LIVE, ON STAGE

We swap the address

Repoint the seller's registry entry at an attacker, then run the demo again.

The seller still asks for its real address. The numbers disagree — the agent walks away.

A referee, not a cashier

On other chains the facilitator pulls the buyer's funds. On MOI only you can move your own money — so the buyer pays itself, and the facilitator proves it happened.

ELSEWHERE

The token contract verifies an off-chain signature, so a third party can move your funds for you. The facilitator holds the money.

HERE

The buyer submits its own transfer and signs a statement naming it. The facilitator reads the chain and confirms. It signs nothing.

Nine checks, all read-only — signature, key ownership, amount, freshness, is the payee a registered agent, did the transfer land, has it been spent before.

- + Eleven forged payments rejected — each for the right reason
- + One honest payment accepted — the control that makes the rest mean anything

Identity today. Authority next.

One idea per session, so each one lands. Today's demo deliberately says nothing about what an agent is allowed to spend — that's a different idea, and it gets its own session.

SESSION 07

Identity + payment

Who am I paying? The registry answers it, and the agent refuses when the answer is wrong. You are here.

SESSION 08

Authority

What may my agent spend? A cap the chain itself enforces. The agent doesn't get to be well behaved — it simply can't.

SESSION 09

Conditional settlement

What if it never arrives? Funds release on delivery. Escrow as a chain primitive, not a trusted middleman.

You can't give authority to an agent that has no identity. Today we spent that identity.

Run it in sixty seconds

Fund one devnet wallet, run two setup scripts, and the same demo settles for real on Voyage — with an interaction hash you can look up.

```
terminal
```

```
git clone .../MOI-Webinars && cd MOI-Webinars/session-7  
pnpm install
```

```
pnpm setup:asset && pnpm setup:registry    # one-time  
pnpm demo                                # the happy path  
pnpm demo -- --tamper                     # watch it refuse
```

ONE WALLET IS ENOUGH

Only the buyer needs funding — the seller receives, so it never signs and never needs gas.

READ THE CODE

Start at `verify-payment.ts` — the nine checks are the session in one file.